



Lecture 01: The Compiler Architecture

INTERVIEW QUESTION

What is Svelte, and how does its architectural approach differ from Virtual DOM-based frameworks like React or Vue?

Every web framework promises to make building user interfaces easier, but beneath the surface, they must all solve the same fundamental problem: when the data changes, how do we update the browser's Document Object Model (DOM) efficiently?

To understand Svelte, we must first understand the problem it was created to solve: the overhead of the Virtual DOM.

The Problem: The Runtime Tax

In traditional frameworks like React or Vue, the framework itself is a large library of JavaScript that you ship to the user's browser. This library acts as an intermediary between your code and the actual DOM.

When a user clicks a button and changes a piece of state, the framework does not immediately update the screen. Instead, it performs a multi-step process:

1. It runs your component's render function to build a complete **Virtual DOM**—a giant JavaScript object that represents what the UI *should* look like.
2. It compares (or "diffs") this new Virtual DOM against a snapshot of the previous Virtual DOM.
3. It figures out exactly which physical DOM nodes need to change.
4. Finally, it applies those surgical changes to the real DOM.

This approach is highly effective, but it carries a "runtime tax." The browser is forced to download the framework's reconciliation engine, and the user's device must spend CPU cycles constantly building and diffing these massive Virtual DOM trees on every single interaction.

The Mechanism: Shifting Work to Compile Time

Svelte fundamentally changes where this work happens. Rather than shipping a heavy runtime engine to the browser to figure out what changed, **Svelte is a compiler**.

When you build your application for production, the Svelte compiler parses your declarative `.svelte` components—which contain standard HTML, CSS, and JavaScript. Because it analyzes your code at compile time, Svelte can see exactly how your state variables map to your markup.

Instead of outputting Virtual DOM structures, the compiler generates lean, imperative, vanilla JavaScript. This generated code contains the precise instructions needed to update the exact DOM nodes that rely on your state. There is no Virtual DOM, no diffing engine, and no heavy runtime to download.

The Result: Surgical DOM Updates

When state changes in a Svelte application, the compiled code does not need to pause and calculate what changed. It already knows. It acts surgically, updating only the specific text node or attribute that depends on that state.

Let's look at how this works in practice.



```
1  <script>
2    let count = $state(0);
3
4    function increment() {
5      count += 1;
6    }
7  </script>
8
9  <button onclick={increment}>
10    Clicked {count} times
11  </button>
```

The compiler tracks this **STATE** automatically

Svelte wires the event directly to the **DOM**

In a Virtual DOM framework, clicking this button triggers a re-render of the component tree to figure out that the number inside the button changed.

In Svelte, the compiler statically analyzes the `{count}` expression inside the button. It generates a tiny piece of JavaScript that effectively says: *When `count` changes, directly update the text content of this specific button element.*

What "Compiler" Actually Means

A common point of confusion is what a compiler means practically for your web application.

Think of traditional frameworks (React or Vue) like a child riding a bicycle with training wheels. To ride the bike, the training wheels must **always** be attached. The framework ships *two* things to the browser: your custom code (the bike) AND the framework engine necessary to run it (the training wheels). Your code cannot run autonomously; it requires the framework to be present in "live performance" to figure out what to update.

Because Svelte is a compiler, it acts as the process of taking the training wheels off before the user ever sees the bike.

Traditionally in computer science, a compiler is a tool that translates human-written code into a **standalone executable file** (like a `.exe` on Windows or a `.app` on macOS). A user just clicks the `.exe` and runs the program, without needing to install or run the massive software framework that originally created the executable.

Svelte brings this exact philosophy to the web. The compiler is a tool that **gets rid of the framework**. It translates your `.svelte` files into a single, independent JavaScript file that lives autonomously in the browser. You don't ship a "Svelte Engine" to the user; you just ship the final, standalone executable code.

Summary

Svelte's architectural approach shifts the heavy lifting from the browser (runtime) to the build step (compile time). By compiling declarative components into surgical, imperative DOM updates, Svelte eliminates the need for a Virtual DOM. This results in smaller bundle sizes, faster initial load times, and highly efficient updates, as the browser executes only the exact code needed to keep the UI in sync with your data.

React	<code>.jsx</code> or <code>.tsx</code> files	Bundled JS + React Framework Engine
Vue	<code>.vue</code> files	Bundled JS + Vue Framework Engine
Svelte	<code>.svelte</code> files	ONLY compiled vanilla <code>.js</code> (No Framework)

Lecture 01b: The Abstract Syntax Tree (AST) Deep Dive

INTERVIEW QUESTION

What is an Abstract Syntax Tree (AST), and how does Svelte's use of ASTs differ from how React and Vue use them?

(Note: This is an advanced conceptual question. Knowing the answer elevates you from a standard "framework user" to a developer who understands computer science fundamentals.)

When we talk about a compiler "parsing" your code, we are actually talking about the creation of an **Abstract Syntax Tree (AST)**. An AST is a universal concept in computer science. Almost every programming language, compiler, or interpreter relies on it.

When you write code, you see it as a flat string of text (e.g., `let x = 5;`). But a computer cannot reason about a string of text. To process your code, a parser breaks that text down into a giant, branching tree structure of objects.

KEY TAKEAWAY

What the DOM is to HTML, the AST is to code.

Just as the browser parses a flat HTML string into a Document Object Model (DOM) tree so it can manipulate the web page, a compiler parses a flat string of code into an Abstract Syntax Tree (AST) so it can analyze and manipulate the program.

For example, `1 + 2` becomes a tree where:

- The top "Node" is a `BinaryExpression` (Addition)
- The left "Branch" is a `NumericLiteral` (1)
- The right "Branch" is a `NumericLiteral` (2)

This tree structure allows a program to systematically walk through your code, understand its logic, and translate it into something else.

Do React and Vue use ASTs?

Yes, they absolutely do! But they use them very differently than Svelte.

1. React

When you write React, you write JSX (like `<h1>Hello</h1>`), which is not valid JavaScript. Before your code hits the browser, a build tool (like Babel or SWC) uses an AST to translate that JSX into standard JavaScript functions (like `React.createElement('h1', null, 'Hello')`).

However, that is where React's compile-time work stops. At *runtime* in the browser, React must rely on the **Virtual DOM** to figure out what data changed and what physical pixels to update.

2. Vue

Vue works similarly. A Vue compiler reads your `.vue` templates, builds an AST of your HTML, and translates it into JavaScript render functions. But just like React, Vue still relies

heavily on a Virtual DOM at runtime to apply those updates.

The Svelte Difference

React and Vue use ASTs primarily to *transpile their syntax* into a format the browser can run.

Svelte goes much deeper. Svelte uses the AST to fundamentally **map out the logic of your application**.

Because Svelte traces exactly where a variable is declared and exactly what HTML element it touches inside the AST, it can skip the Virtual DOM entirely. It uses the AST to write the exact, direct DOM update instructions (e.g., `h1.textContent = new_value`) straight into the final standalone JavaScript bundle.

INTERVIEW TIP

To impress the interviewer: During an interview, most candidates will simply state that "Svelte doesn't use a Virtual DOM." If you want to show deep technical background and skills, explain *why* it doesn't need one: *"Svelte doesn't need a Virtual DOM at runtime because it uses an Abstract Syntax Tree at compile-time to permanently map the relationship between state and the DOM."* This proves you understand the underlying mechanics of compilers and aren't just reciting marketing material.

Summary

An AST is a tree representation of your code's structure. While React and Vue use ASTs simply to translate custom syntax into Virtual DOM instructions, Svelte uses the AST to permanently trace reactivity and generate direct DOM updates, completely eliminating the need for a runtime diffing engine.

React	Transpiles JSX into JS functions	Virtual DOM diffing engine
Vue	Transpiles templates into JS render functions	Virtual DOM diffing engine
Svelte	Maps state directly to HTML elements	Direct DOM updates (No Virtual DOM)

Lecture 02: Inside the Build Step

INTERVIEW QUESTION

Explain what it means that Svelte is a compiler. What happens during the build step?

In the previous lecture, we established that Svelte relies on a compiler rather than a runtime engine. But "compiler" is a broad term. What exactly happens between the moment you save a `.svelte` file and the moment your application runs in the browser?

Browsers do not natively understand `.svelte` files. A `.svelte` file is a custom authoring format that elegantly combines JavaScript, HTML, and CSS into a single component. The Svelte compiler is a Node.js program that translates this custom format into standard web assets (`.js` and `.css` files) that a browser can execute.

When you run your build command (typically via Vite or Rollup), the compiler processes your components in three main phases.

Phase 1: Parsing and Analysis

First, the compiler reads your `.svelte` file and breaks it down into an **Abstract Syntax Tree (AST)**—a structured map of your code.

During this phase, it analyzes the relationships between your data and your user interface. It looks for Svelte-specific reactivity markers (like the `$state` rune) and scans your HTML markup to see exactly where those reactive variables are used. By building this map at compile time, Svelte figures out exactly what DOM nodes will need to be updated when a specific variable changes.

Phase 2: CSS Extraction and Scoping

Next, the compiler processes your `<style>` blocks. In Svelte, styles are "scoped" to the component by default, meaning a `p` tag style in one component won't accidentally leak and alter a `p` tag in another component.

The compiler achieves this by generating a unique hash string (e.g., `svelte-1xyz59`) for your component. It then automatically appends this hash as a class name to both your HTML elements and your CSS selectors. Finally, it extracts all the CSS from all your components and bundles it into a separate, highly optimized `.css` file.

Phase 3: JavaScript Generation

In the final phase, the compiler strips away the HTML and Svelte-specific syntax, generating pure, vanilla JavaScript.

To understand what is generated, we first need to understand the **Component Lifecycle**. In any modern web framework (Svelte, React, Vue), you must pay attention to when a component suddenly appears on your screen (Mounted), when it changes (Updated), and when it vanishes from the screen (Destroyed). Why? Because as a developer, you often need to run specific code at specific times. For example, you might need to make sure external data is fetched *before* a component is mounted, or you must ensure that memory leaks (like active timers or open websockets) are halted when a component is destroyed.

The Svelte compiler produces a set of imperative instructions that perfectly handle this complete lifecycle for you:

1. **Mounting:** Instructions to create the physical DOM nodes (`document.createElement`).
2. **Updating:** The precise instructions to update *only* the specific text nodes or attributes when the tracked state changes.
3. **Destroying:** Instructions to clean up the DOM and remove event listeners when the component is unmounted.

Let's look at a conceptual example of what this transformation looks like.

```
App.svelte
1  <script>
2    let name = $state('World');
3  </script>
4
5  <h1>Hello {name}!</h1>
6
7  <style>
8    h1 { color: red; }
9  </style>
```

When the compiler finishes with this file, the browser never sees the `<h1>` tag or the `<style>` block. Instead, the compiler generates separate, highly-optimized assets that look conceptually like this:

```
Conceptual_Compiled_Output.js
1  const h1 = document.createElement('h1');
   1. the DOM element is created via JavaScript
2  h1.className = 'svelte-xyz123';
   2. CSS is SCOPED with a hash and extracted to bundle.css
3  function update_name(new_name) {
   3. the exact update instruction is HARDCODED
4    h1.textContent = `Hello ${new_name}!`;
5  }
```

INTERVIEW TIP

To impress the interviewer: When discussing the build step, don't just say "it compiles to JS." Emphasize that the compiler translates your **declarative** component into **imperative** DOM instructions (like `document.createElement`). Showing that you understand the difference between declarative authoring and imperative execution is a massive green flag for senior engineering roles.

Summary

KEY TAKEAWAY

The Svelte build step fundamentally does three things: parses the AST to map reactivity, extracts scoped CSS, and generates imperative vanilla JavaScript instructions.

Saying "Svelte is a compiler" means that Svelte does its heaviest lifting during the build step. It parses your `.svelte` files, scopes and extracts your CSS, and translates your declarative markup and state into surgical, vanilla JavaScript instructions. Because this analysis happens at compile time, the final code shipped to the browser is lean, fast, and completely stripped of framework overhead.

1. Parsing & Analysis	Reads <code>.svelte</code> file, builds AST, and maps reactivity.	Dependency map of state-to-markup relationships.
2. CSS Extraction	Scopes styles with a unique hash (e.g. <code>svelte-xyz</code>).	A highly optimized, standalone <code>.css</code> bundle.

3. JS Generation

Strips HTML/
Runes, writes
imperative
DOM code.

Lean, vanilla `.js`
instructions for
component
mounting and
updating.

Lecture 03: The Performance Benefits of a Compiler

INTERVIEW QUESTION

What are the primary performance benefits of shifting work from runtime to compile time?

We have established that Svelte is a compiler (Lecture 01) and we have seen how it translates declarative code into pure JavaScript during the build step (Lecture 02). But why did Svelte's creators go to the trouble of building an entire compiler? What is the tangible, real-world payoff for the end user?

Shifting the heavy lifting from the browser's runtime environment to the developer's compile-time environment yields three massive performance advantages.

1. Minimal Bundle Size (The Network Benefit)

Every byte of JavaScript sent over the network must be downloaded, parsed, and executed by the browser. Traditional frameworks require you to ship the framework's runtime library (often 30-100kb of minified, zipped code) alongside your actual application code.

Because Svelte does its work at compile time, there is no heavy "Svelte library" to send. The compiler generates only the specific vanilla JavaScript needed to make your exact components work. For a small or medium-sized application, this results in drastically smaller bundle sizes, meaning faster downloads and quicker "Time to Interactive" metrics for users on slow mobile networks.

2. Surgical Updates (The CPU and Battery Benefit)

To understand Svelte's biggest performance advantage, we need to ask: **What is the most efficient way to update the screen when data changes?**

Is it better to wait until the user clicks a button, generate an entire Virtual DOM tree, generate a *second* Virtual DOM tree, and then algorithmically diff them to figure out what changed? Or is it better to map out all those connections *before* the user even loads the page?

KEY TAKEAWAY

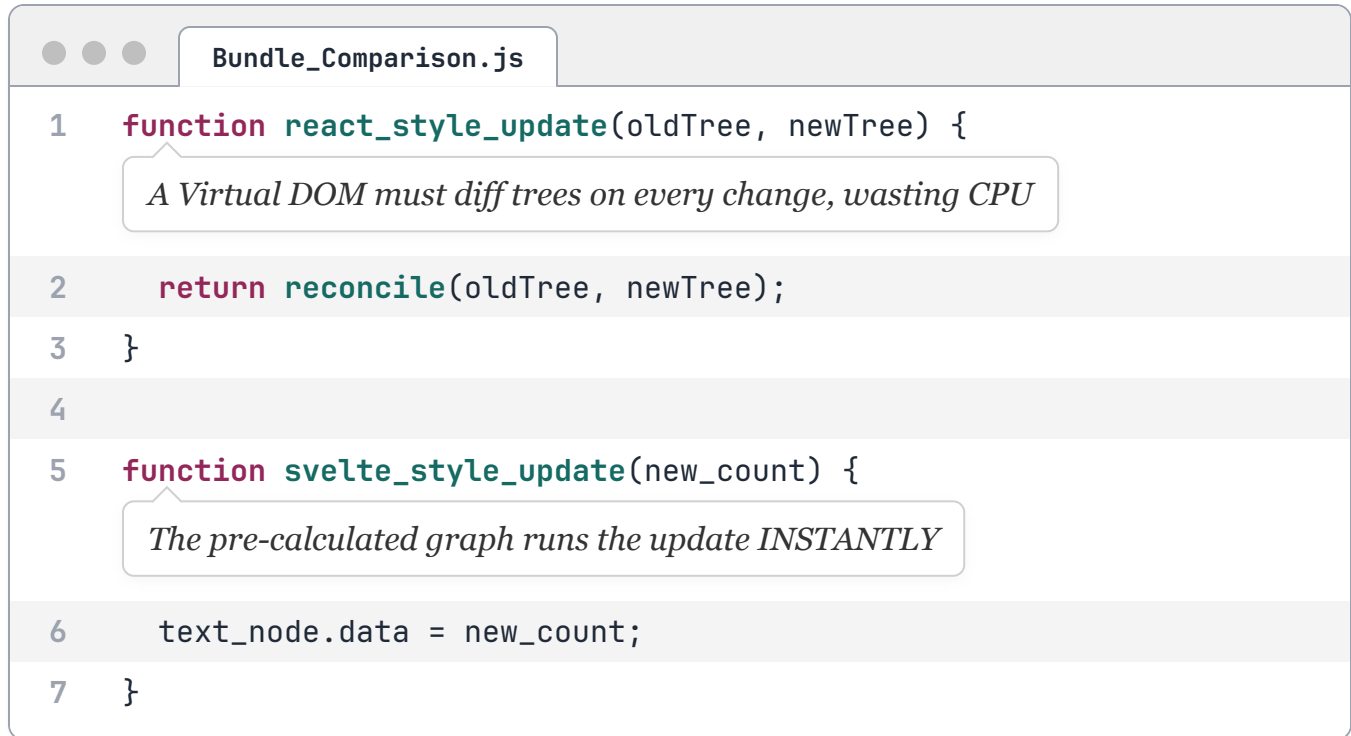
Svelte solves the UI as a mathematical logic problem at compile time (Dependency Graph), while traditional frameworks solve it by playing "spot the difference" at runtime (Virtual DOM Diffing).

Remember: **What the DOM is to HTML, the AST is to the logic of the code.** A traditional framework ships its runtime code to the browser to do DOM calculations. But Svelte is a compiler. It does its advanced calculations at compile time, using the AST.

Because Svelte parses your `.svelte` components into an AST at build time, it can use that mathematical tree of logic to construct a **Dependency Graph** (also known as a Reactivity Graph).

You can think of this Dependency Graph as the pre-calculated "mind" of the application. It maps out the invisible connections between every piece of your logic. If you click a button that affects a completely different component on the other side of the tree, Svelte already knows about that built-in relationship from the graph. It doesn't need to recalculate or diff the UI tree again and again—it just follows the graph directly to the target.

By calculating this graph at build time, Svelte eliminates the need for a Virtual DOM entirely. The compiler simply hardcodes the exact DOM update instructions right into the component.



```
1  function react_style_update(oldTree, newTree) {  
    A Virtual DOM must diff trees on every change, wasting CPU  
2  return reconcile(oldTree, newTree);  
3  }  
4  
5  function svelte_style_update(new_count) {  
    The pre-calculated graph runs the update INSTANTLY  
6    text_node.data = new_count;  
7  }
```

When state changes, Svelte doesn't have to think, diff, or reconcile. It just follows the pre-calculated Dependency Graph and executes a surgical `text_node.data = new_value` assignment. This requires virtually zero CPU overhead, leading to perfectly smooth 60fps animations and preserving battery life on mobile devices.

3. Aggressive Dead Code Elimination

Because a compiler analyzes your entire codebase holistically before outputting the final build, it knows exactly what you are *not* using.

If you import a massive CSS library but only use two classes, the Svelte compiler can safely strip out the rest. If you write a helper function but never call it, the compiler removes it. This level of aggressive "tree-shaking" is much harder for a runtime framework to achieve perfectly, because a runtime framework can't always predict how you might use its library dynamically.

INTERVIEW TIP

To impress the interviewer: When asked about Svelte's performance, most candidates will simply say "it's faster because there's no Virtual DOM." To stand out, break the performance benefits down into their three distinct categories: **Network time** (smaller bundles), **CPU/Battery usage** (no Virtual DOM diffing), and **Memory** (aggressive dead code elimination).

Summary

By moving the hardest work to compile time, Svelte shields the user from the cost of the framework. The primary performance benefits are much smaller bundle sizes (saving network time), surgical DOM updates without Virtual DOM diffing (saving CPU and battery), and perfect dead code elimination. The developer gets the convenience of a modern framework, while the user gets the performance of hand-written, optimized vanilla JavaScript.

	SVELTE (Compile-Time)	REACT / VUE (Runtime)
Bundle Size	Minimal (ships only your code)	Large (ships the framework library + your code)
Update Mechanism	Surgical DOM updates via a pre-calculated Dependency Graph	Virtual DOM diffing and reconciliation
CPU/Battery Cost	Very low (runs instantaneous, hardcoded)	High (diffs massive UI trees in memory on

	updates)	every interaction)
Dead Code	Aggressively stripped during the build	Often shipped to the user because runtime usage is unpredictable

Lecture 04: Surgical DOM Updates Without a Virtual DOM

INTERVIEW QUESTION

How does Svelte achieve surgical DOM updates without tracking changes via an in-memory Virtual DOM tree?

In the previous lecture, we established that Svelte pre-calculates a Dependency Graph at compile time. But how does that graph actually translate into changing a specific `<h1>` tag in the browser without tearing down the whole page?

To understand Svelte's surgical precision, we must first understand the bottleneck it was designed to bypass: the Virtual DOM.

The Problem with "Spot the Difference"

When state changes in a React application, the framework doesn't inherently know which specific HTML element needs to change. Instead, it runs the component function again and generates a **Virtual DOM**—a lightweight, in-memory representation of what the UI *should* look like.

It then compares this new Virtual DOM tree against the old Virtual DOM tree. This process, called **Reconciliation**, involves algorithmically traversing the tree to play "spot the difference." Once it finds the difference, it finally updates the real DOM.

This works, but it scales poorly. If you change a single text node deep inside a massive table, the framework still has to diff the entire table's Virtual DOM to find that one tiny change. It is pure overhead.

The Svelte Solution: Direct Signal Wiring

Svelte takes a fundamentally different approach. It discards the Virtual DOM entirely. Instead of comparing the *structure of the UI* to find what changed, Svelte connects the *data itself* directly to the specific DOM node that relies on it.

In Svelte 5, this is achieved using **Runes** (like `$state`), which act as reactive signals under the hood. When the compiler parses your component, it identifies exactly where a piece of `$state` is used in the HTML. It then wires up a direct, microscopic subscription—an effect—between that specific piece of state and that specific DOM element.

KEY TAKEAWAY

A Virtual DOM diffs the UI to find the data. Svelte signals wire the data directly to the UI.

When the data changes, the signal instantly notifies its exact subscribers. Svelte does not need to traverse a tree or compare old states to new states; it simply executes the tiny, hardcoded function that updates that exact DOM node.

The Evolution of the Dependency Graph

In older versions of Svelte (v3/v4), the Dependency Graph was entirely static. The compiler would analyze your code during the build step and physically order the JavaScript instructions so that updates ran in the correct sequence. It was fast, but it struggled with complex conditional logic because it tried to predict the future before the code even ran.

Svelte 5 solves this by turning the Dependency Graph into a **Signal-Based Architecture**.

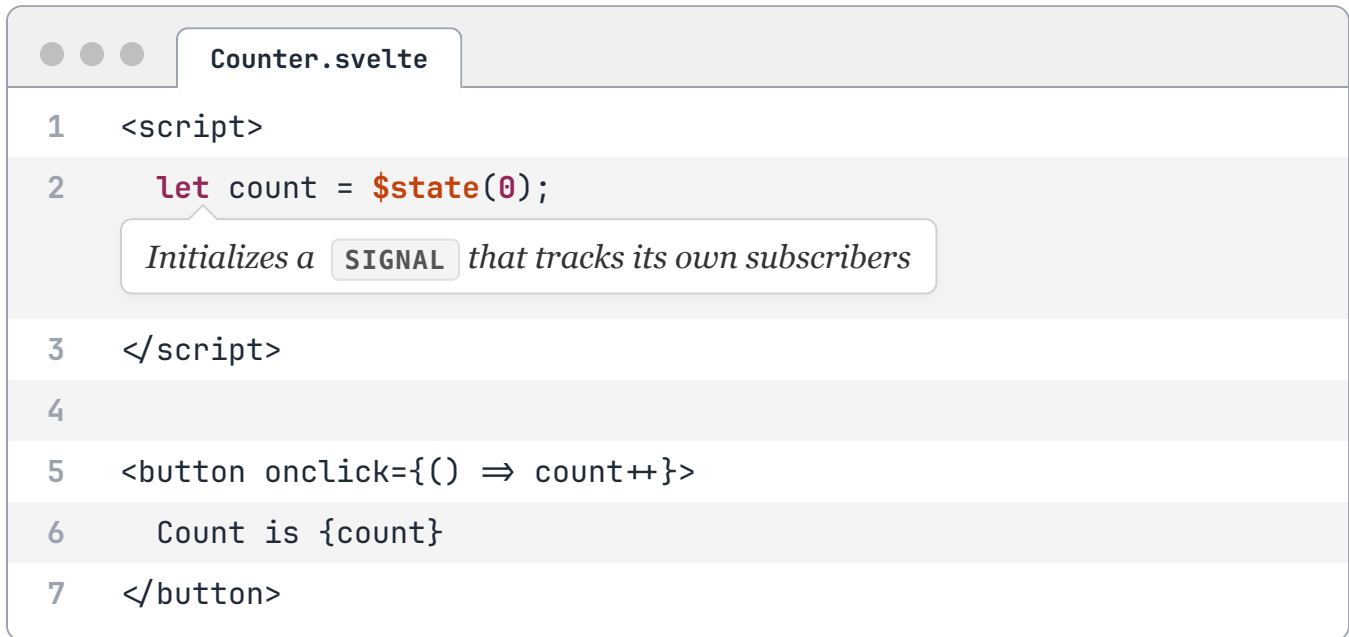
A Signal architecture is simply a Dependency Graph that is maintained at *runtime* instead of *compile-time*:

- A **Signal** (like `$state`) is a node in the graph that holds data.
- A **Subscriber** (like `$effect`) is a node that listens to that data.

Instead of the compiler hardcoding the exact order of updates during the build, the compiler's job is now just to set up the DOM nodes and wrap them in tiny subscriptions. When the page loads, those Signals instantly wire themselves together in the browser, building a dynamic Dependency Graph on the fly.

A Worked Example

Consider a simple counter component. Let's look at how Svelte compiles it into surgical DOM operations.



```
1  <script>
2    let count = $state(0);
3  </script>
4
5  <button onclick={() => count++}>
6    Count is {count}
7  </button>
```

When you click the button and `count` increments, Svelte does *not* re-render the button or the component. Because the compiler saw that `{count}` was used inside the text node of the button, it compiled that specific binding into a reactive effect.

Behind the scenes, the compiled JavaScript looks something like this:

● ● ● compiled_output.js

```
1  export function Counter() {
2    let count = $.state(0);
3
4    var button = $.element('button');
5    var text = $.text(button);
6
7    $.effect(() => {
8      $.set_text(text, `Count is ${$.get(count)}`);
9    });
10 }
```

The data is wrapped as a reactive SOURCE

The DOM element is created ONCE

The text node is isolated as a TARGET

Updates DIRECTLY

Notice what is missing: there is no `render()` function that runs every time `count` changes. There is no Virtual DOM diffing. The `$.effect` is a direct subscription to the `count` signal. When `count` changes, only that specific `$.set_text` instruction runs, surgically updating the single text node in the browser.

The Advantage of Signals

This granular reactivity is what makes Svelte "surgical." The framework never wastes CPU cycles guessing what changed. It knows exactly what changed, and it knows exactly which DOM node to update, making it wildly efficient even in massive applications with thousands of reactive variables.

INTERVIEW TIP

To impress the interviewer: Interviewers often ask this to test your historical knowledge of frameworks. A great answer will explicitly mention that while older frameworks relied on Virtual DOM reconciliation, Svelte 5 uses a modern **Signal-based architecture** under the hood, generated at compile-time, to achieve fine-grained reactivity.

Summary

While React and Vue rely on a Virtual DOM to diff and reconcile changes at runtime, Svelte takes a fundamentally different path. By using a modern, Signal-based architecture, Svelte wires data directly to the DOM nodes that depend on it. When state changes, there are no trees to traverse or differences to spot—Svelte simply executes a hardcoded instruction to update exactly what changed, resulting in unmatched efficiency and surgical precision.

Feature	SVELTE (Signals / Compile-Time)	REACT / VUE (Virtual DOM / Runtime)
Update Strategy	Direct subscriptions (Signals notify DOM nodes)	Reconciliation (Tree diffing)
CPU Overhead	Near zero (runs predefined update instructions)	High (traverses and compares objects in memory)
Memory Usage	Low (no duplicate UI)	High (stores multiple Virtual

	trees stored in memory)	DOM trees)
Component Executions	Runs exactly once during creation	Runs repeatedly on every state change

Lecture 05: The Anatomy of a Svelte Component

INTERVIEW QUESTION

Describe the typical anatomy and structure of a single-file .svelte component.

Now that we understand Svelte is a compiler that produces surgically precise DOM updates, it is time to look at the source material: the `.svelte` file itself.

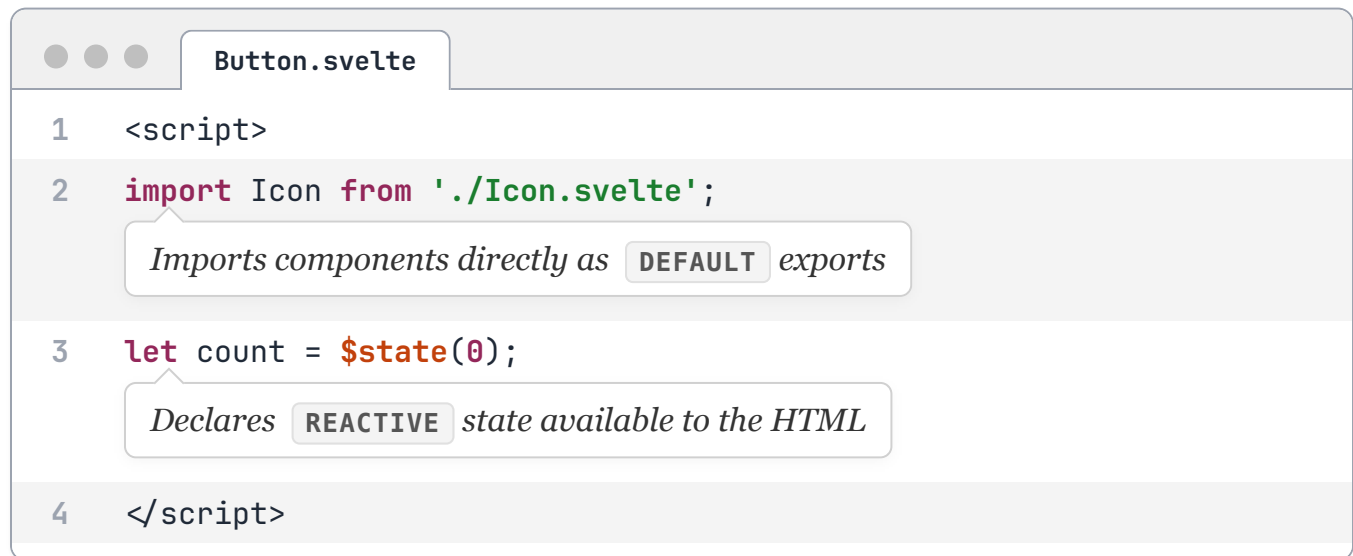
If you have used Vue, the concept of a Single-File Component (SFC) will be very familiar. Svelte embraces the SFC pattern, meaning a single `.svelte` file encapsulates everything a component needs to function: its logic, its styling, and its markup.

The structure of a `.svelte` file is incredibly simple and closely mirrors standard HTML. There is no boilerplate, no `export default class`, and no required `render()` method wrapper. It is broken into three distinct sections. The community standard usually places `<script>` at the top, followed by the markup, and `<style>` at the bottom.

1. The Logic (`<script>`)

At the top of the file, you place your JavaScript (or TypeScript) inside a standard `<script>` tag.

This block runs exactly once when the component is instantiated. It is where you declare your reactive state (using `$state`), define your functions, and import other components or libraries. Any variable declared at the top level of this script is automatically made available to the markup below it.



The screenshot shows a code editor window titled "Button.svelte". The code is as follows:

```
1 <script>
2 import Icon from './Icon.svelte';
3 let count = $state(0);
4 </script>
```

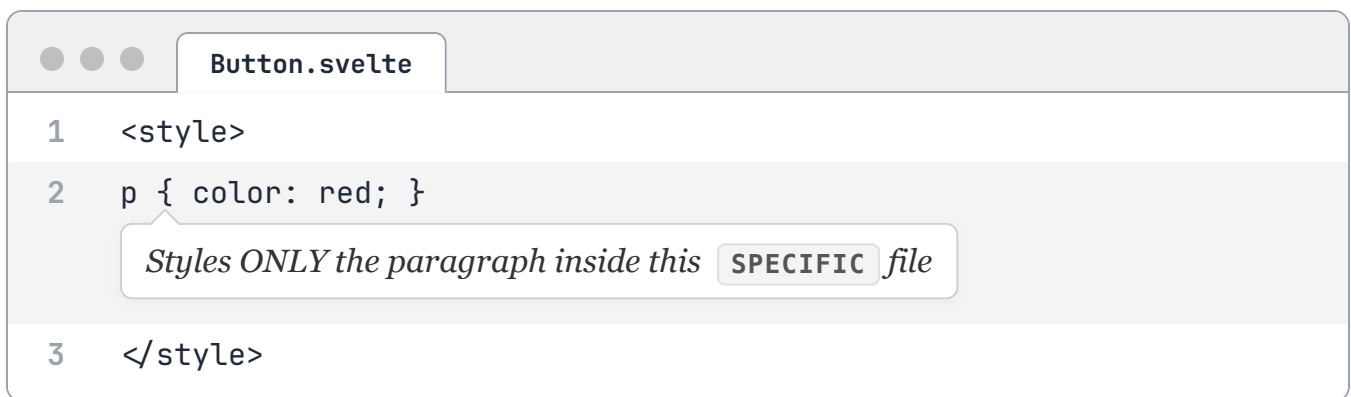
Two callout boxes provide additional context:

- A callout box pointing to line 2 states: "Imports components directly as **DEFAULT** exports".
- A callout box pointing to line 3 states: "Declares **REACTIVE** state available to the HTML".

2. The Styling (`<style>`)

You write your CSS inside a standard `<style>` tag.

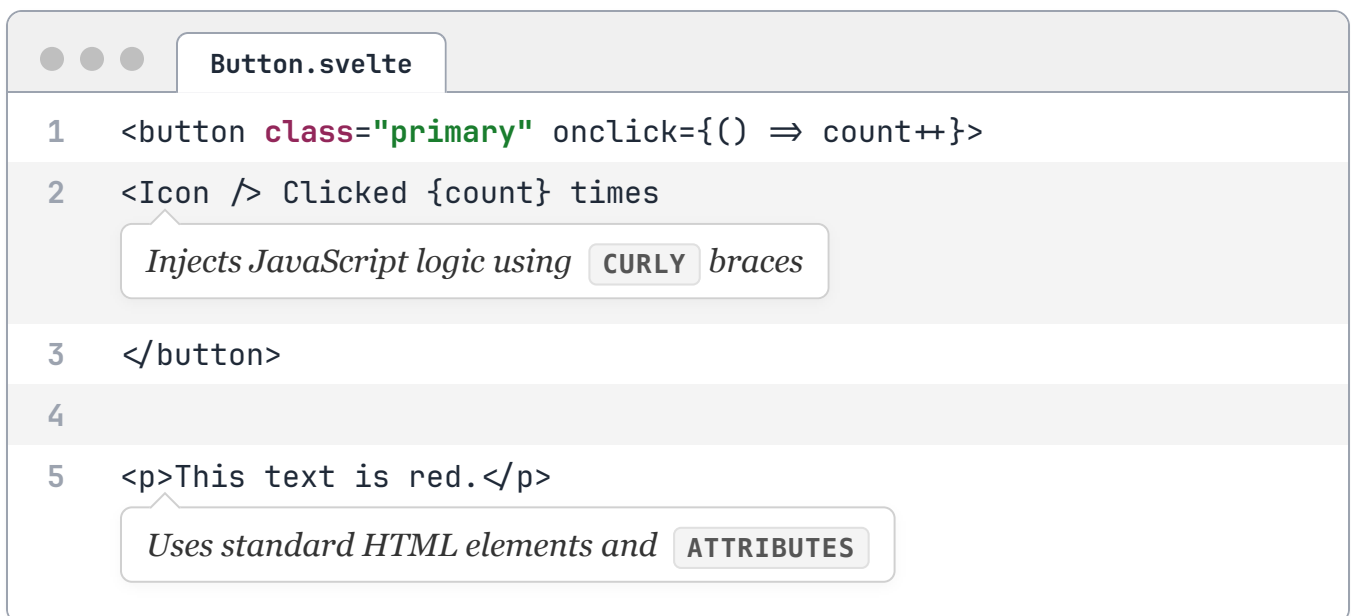
The magic of Svelte's compiler is that all styles are scoped by default. If you write a rule targeting a `p` tag in `Button.svelte`, it will only style `<p>` tags *inside* that specific component. It will not leak out and accidentally ruin the styling of paragraphs elsewhere in your application. The compiler enforces this by hashing a unique class name and applying it to both the CSS and the HTML during the build step.



3. The Markup (HTML)

Everything else in the file is treated as the component's markup. Svelte uses standard HTML augmented with a powerful, specialized templating syntax. You can use single curly braces `{}` to inject JavaScript expressions directly into the DOM or to bind attributes.

Because Svelte embraces standard HTML, you don't need to learn JSX quirks (like using `className` instead of `class` or `htmlFor` instead of `for`). It is just HTML.



```
1 <button class="primary" onclick={() => count++}>
2 <Icon /> Clicked {count} times
3 </button>
4
5 <p>This text is red.</p>
```

*Injects JavaScript logic using **CURLY** braces*

*Uses standard HTML elements and **ATTRIBUTES***

KEY TAKEAWAY

A Svelte component is just an HTML file with superpowers. The `<script>` handles logic, the `<style>` handles scoped presentation, and the rest is standard HTML markup.

Why this architecture wins

By colocating logic, styling, and markup into a single file without wrapping them in complex JavaScript boilerplate, Svelte drastically lowers the cognitive load required to read a component. A new developer can open a `.svelte` file and instantly understand what it does and how it looks, top to bottom.

INTERVIEW TIP

To impress the interviewer: When asked about Svelte's component anatomy, emphasize that Svelte's `<style>` block provides **zero-configuration scoped CSS**. In React, you often have to install and configure third-party libraries (like CSS Modules or Styled Components) to achieve scoping. In Svelte, it is a native feature of the compiler.

Summary

The `.svelte` file is a beautifully simple, standard-compliant approach to building UI components. It requires zero boilerplate, uses native HTML tags, and enforces style isolation automatically, giving developers a highly ergonomic authoring experience.

Section	Purpose	Key Characteristic
<code><script></code>	Component logic and state	Runs ONCE upon component instantiation
<code><style></code>	Component presentation	CSS is automatically SCOPED to the file
Markup	Component structure	Uses standard HTML and <code>{}</code> for LOGIC injection

Lecture 06: Zero-Config Scoped Styling

INTERVIEW QUESTION

How does Svelte enforce scoped styling out of the box for component markup?

One of the most persistent headaches in traditional web development is the global nature of CSS. If you declare `h1 { color: red; }` in a stylesheet, every `<h1>` on your site turns red. To solve this, developers historically relied on strict naming conventions (like BEM) or heavy third-party libraries (like CSS Modules or Styled Components) to keep styles isolated.

Svelte eliminates this problem entirely at the compiler level. In Svelte, **all CSS is scoped to the component by default.**

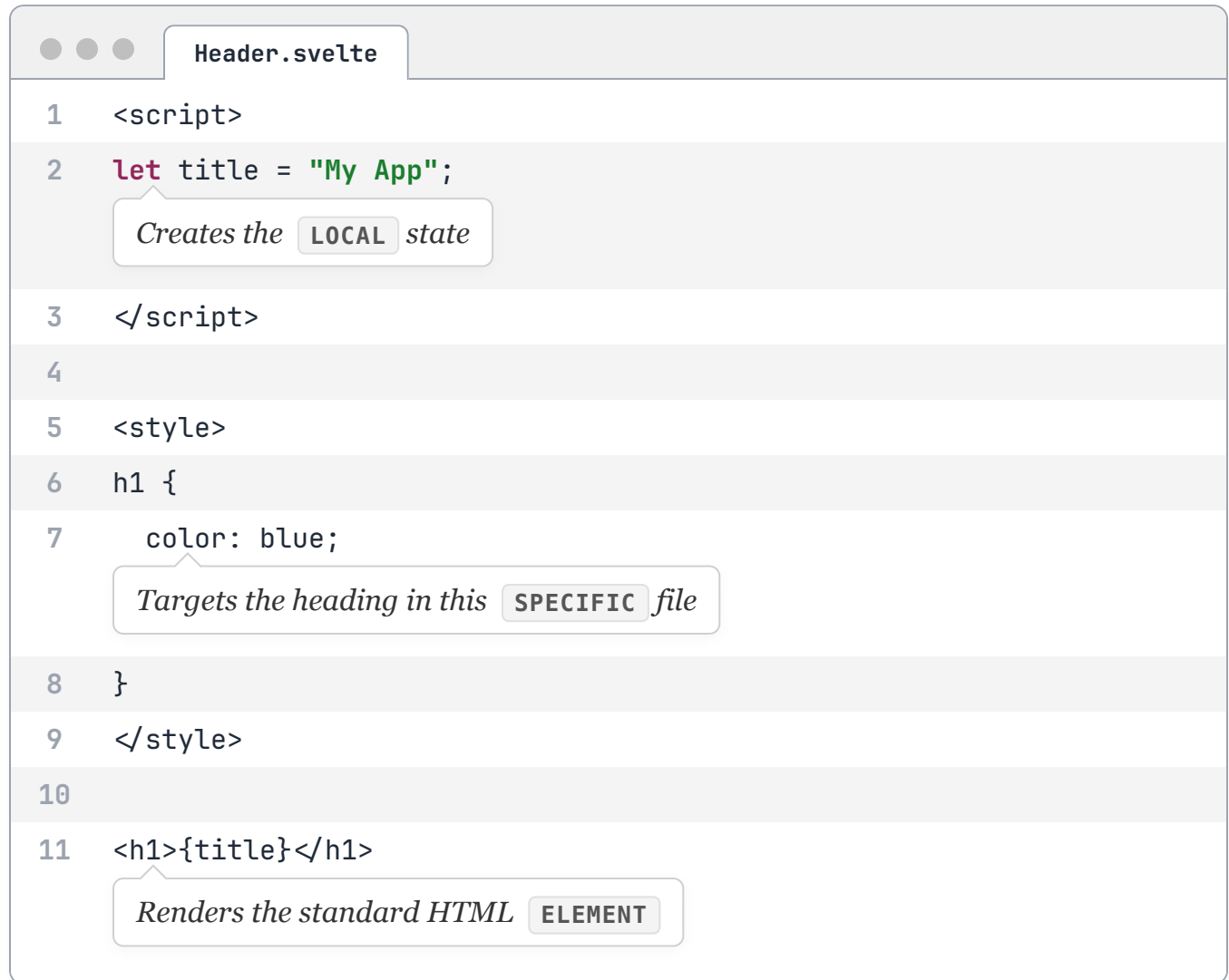
How the Compiler Scopes CSS

When you write a CSS rule inside the `<style>` block of a `.svelte` file, the Svelte compiler performs two simultaneous transformations during the build step:

1. **It generates a unique hash:** The compiler creates a short, unique string (like `svelte-xyz123`) for that specific component.
2. **It mutates the HTML and CSS:** It appends that hash as a class name to the relevant HTML elements, and rewrites your CSS selectors to target that specific hash.

A Worked Example

Consider a simple `Header.svelte` component that attempts to style an `<h1>` element.



```
1  <script>
2  let title = "My App";
3  </script>
4
5  <style>
6  h1 {
7      color: blue;
8  }
9  </style>
10
11 <h1>{title}</h1>
```

The screenshot shows a code editor window titled "Header.svelte". The code is as follows:

- Line 1: `<script>`
- Line 2: `let title = "My App";` (An annotation points to `let`: "Creates the **LOCAL** state")
- Line 3: `</script>`
- Line 4: (empty line)
- Line 5: `<style>`
- Line 6: `h1 {`
- Line 7: `color: blue;` (An annotation points to `h1`: "Targets the heading in this **SPECIFIC** file")
- Line 8: `}`
- Line 9: `</style>`
- Line 10: (empty line)
- Line 11: `<h1>{title}</h1>` (An annotation points to `<h1>`: "Renders the standard HTML **ELEMENT**")

If you look at the raw HTML output in the browser's developer tools after the application is built, you will not see a generic `<h1>` . Instead, you will see exactly how the compiler enforced the scoping:

● ● ●

Compiled Output (Browser)

1 <h1 class="svelte-xyz123">My App</h1>

Compiler injects the generated HASH class

And if you inspect the compiled CSS bundle that Svelte ships to the browser, you will see that your original `h1` selector was rewritten:

● ● ●

bundle.css

1 h1.svelte-xyz123 {

Selector is restricted by the HASH class

2 color: blue;

3 }

KEY TAKEAWAY

Svelte scopes CSS by automatically hashing class names and attaching them to both your HTML elements and your CSS selectors at compile time. It guarantees your styles will never leak.

The Benefits of Native Scoping

Because scoping is handled natively by the compiler, you never have to configure Webpack plugins for CSS Modules, or ship a heavy CSS-in-JS runtime library to the browser. You simply write plain, standard CSS.

Furthermore, you don't need to invent complex, verbose class names like `.header__title--blue` just to avoid collisions. You can safely use bare element selectors like `h1` or `p`, knowing they will only affect the current file.

INTERVIEW TIP

To impress the interviewer: Mention that because Svelte handles styling at compile time, it also performs **dead CSS elimination**. If you write a CSS rule in a component but never actually render the corresponding HTML element, Svelte will issue a compiler warning and safely strip the unused CSS from the final bundle.

Summary

Svelte provides zero-configuration, native CSS scoping out of the box. By relying on the compiler to rewrite selectors and inject unique hashes during the build step, Svelte guarantees that component styles remain perfectly isolated without the need for external tooling, runtime libraries, or complex naming conventions.

	REACT / VUE (Traditional CSS)	SVELTE (<code><style></code>)
Scope	Global by default	SCOPED by default
Tooling Required	BEM, CSS Modules, Styled Components	None (handled by the COMPILER)
Collision Risk	High (styles easily leak)	Zero (unique HASHES prevent leaks)

Lecture 07: Dead CSS Elimination

INTERVIEW QUESTION

What happens to unused CSS selectors defined within a Svelte component's `<style>` block during compilation?

In traditional web development, a phenomenon known as "append-only CSS" is incredibly common. Because developers are terrified of deleting a CSS class and accidentally breaking an unknown page elsewhere in the app, stylesheets grow endlessly over time. Removing unused CSS usually requires configuring complex, external build tools like PurgeCSS.

Because Svelte is a compiler that understands both your markup and your styling simultaneously, it solves this problem natively.

Compile-Time Analysis

During the build step, the Svelte compiler analyzes every HTML element in your component and maps it against the rules defined in your `<style>` block. If the compiler detects a CSS selector that does not match any markup within the component, it takes two specific actions:

1. **It drops the CSS from the bundle:** The unused selector and its associated rules are entirely stripped from the final compiled output. They will not be shipped to the user's browser, keeping your application lean.
2. **It emits a compiler warning:** Svelte surfaces a helpful warning in your terminal and editor (e.g., `Unused CSS selector ".active"`), explicitly informing you of the dead code so you can clean it up.

A Worked Example

Consider a component where a developer wrote an `.active` class, but forgot to actually apply it to the paragraph tag.

● ● ● Status.svelte

```
1 <style>
2   .active {
3     color: green;
4     font-weight: bold;
5   }
6 </style>
7
8 <p>All systems operational.</p>
```

Targets a class that is NEVER used in the markup

Renders WITHOUT the active class

When Svelte compiles this file, it recognizes that `.active` has no target. The output CSS file will be completely empty, and the console will output:

● ● ● Terminal Output

```
1 <style> is unused
2 12: .active {
3     ^
```

KEY TAKEAWAY

Svelte guarantees that no dead CSS from your components will ever reach production. It acts as a safety net, allowing developers to freely delete markup without fear of leaving orphaned styles behind.

INTERVIEW TIP

To impress the interviewer: Contrast Svelte's built-in analysis with traditional frameworks. In React or Vue, standard CSS relies on third-party tools like PostCSS or PurgeCSS to analyze source files and strip unused styles. In Svelte, dead code elimination is a native feature of the framework's compiler architecture, requiring zero configuration.

Summary

When an unused CSS selector is defined in a Svelte component, the compiler catches it during the build step. It safely eliminates the dead CSS from the production bundle, preventing stylesheet bloat, while simultaneously emitting a warning to the developer to clean up their source code.

	STANDARD CSS (Traditional Tooling)	SVELTE (Compile-Time)
Unused Styles	Shipped to the browser by default	Safely STRIPPED from the bundle
Developer Feedback	Silent (no warnings)	WARNINGS emitted in the terminal
External Plugins	Requires PurgeCSS / PostCSS	Built-in (handled by the COMPILER)

Lecture 08: Global CSS in Scoped Components

INTERVIEW QUESTION

How can you intentionally apply global CSS styles from inside a scoped component style block?

While Svelte's default scoped styling is a massive advantage for component isolation, there are times when you genuinely need a CSS rule to break out of its scoped boundary and apply globally.

This typically happens in two scenarios:

1. You need to apply a rule to a top-level HTML element, like `<body>` or `<html>`, directly from a layout component.
2. You are rendering raw HTML from a third-party library (like a Markdown parser) where Svelte's compiler cannot "see" the elements to attach hash classes to them.

To solve this, Svelte provides the special `:global(...)` modifier.

The `:global()` Escape Hatch

When the Svelte compiler encounters the `:global()` modifier in your `<style>` block, it understands that the selector contained within the parentheses should **not** be hashed or scoped to the current component. Instead, it will be left exactly as written in the final CSS bundle, allowing it to target elements anywhere in the DOM.

A Worked Example

Imagine we have a `BlogPost.svelte` component that takes a raw HTML string generated from a Markdown file and injects it into the DOM. Because the Svelte compiler didn't create those `<p>` or `<blockquote>` tags, it didn't give them `svelte-xyz123` classes. If we just write `p { color: gray; }`, Svelte will scope it, and it won't affect the injected HTML.

Instead, we use `:global( )` to escape the scope, but we nest it under a scoped container to ensure we don't accidentally style every paragraph on the entire website.

BlogPost.svelte

```
1  <script>
2  let postContent = "<p>This was parsed from markdown.</p>";
3  </script>
4
5  <style>
6  .article-container :global(p) {
7      color: gray;
8      line-height: 1.6;
9  }
10
11  :global(body) {
12      background-color: #fafafa;
13  }
14  </style>
15
16  <article class="article-container">
17      {@html postContent}
18  </article>
```

Represents dynamic HTML generated at **RUNTIME**

Escapes scope for the paragraph, but keeps the container **SCOPED**

Targets the top-level HTML body across the **ENTIRE** app

Injects raw HTML into the **DOM**

KEY TAKEAWAY

The `:global()` modifier is a deliberate escape hatch. By wrapping a selector in `:global()`, you instruct the compiler to skip its automated hashing process for that specific rule, allowing it to style elements beyond the component's boundaries.

INTERVIEW TIP

To impress the interviewer: Mention the architectural best practices surrounding this feature. While `:global()` is highly useful for styling dynamically injected third-party HTML, truly app-wide styles (like CSS resets, typography baselines, or utility classes) belong in a dedicated `global.css` file imported at the root layout. Overusing `:global()` inside deeply nested components is a code-smell that leads to unpredictable styling collisions.

Summary

The `:global(...)` modifier allows developers to explicitly bypass Svelte's native CSS scoping. It is the framework-provided mechanism for applying global CSS rules from inside a single-file component, and is particularly essential when styling raw HTML injected at runtime that the compiler cannot statically analyze.

	STANDARD SELECTOR (<code>h1 { }</code>)	GLOBAL MODIFIER (<code>:global(h1) { }</code>)
Compiler Action	Injects a unique <code>svelte-hash</code> class	IGNORES the selector (leaves it unchanged)
Reach	Strictly limited to the current file	Affects the ENTIRE DOM tree
Primary Use Case	Styling static component markup	Styling injected HTML (e.g., Markdown parsers)

Lecture 09: Component Logic and the `<script>` Tag

INTERVIEW QUESTION

What is the role of the `<script>` tag in a Svelte component?

In a `.svelte` single-file component, the `<script>` block is the designated home for your component's state, data, and logic. It is where you import dependencies, declare variables, and define functions that your markup will interact with.

However, its most important characteristic is its execution model.

The Initialization Phase

Unlike frameworks like React, where the entire component function re-runs from top to bottom every time a piece of state changes, the code inside a Svelte `<script>` block executes **exactly once** when the component is instantiated.

It acts strictly as an initialization phase. When a component is mounted to the DOM, Svelte runs the `<script>` block to set up the initial state and wire up any event listeners. Once that setup is complete, the `<script>` block never executes again for the lifespan of that specific component instance.

A Worked Example

Consider a standard counter component. The variable declarations and function definitions happen only during creation.

```
1  <script>
2  let count = $state(0);
3
4  function increment() {
5    count++;
6  }
7  </script>
8
9  <button onclick={increment}>
10    Clicked {count} times
11  </button>
```

Counter.svelte

1 <script>

2 let count = \$state(0);

3

4 function increment() {

5 count++;

6 }

7 </script>

8

9 <button onclick={increment}>

10 Clicked {count} times

11 </button>

Initialized exactly ONCE when the component mounts

Declared ONCE and stored in memory

Executes the function on click

When the user clicks the button, `count` increases. In React, this state change would cause the entire component to re-execute to generate a new Virtual DOM tree. In Svelte, the `<script>` block does not re-run. Instead, Svelte's compiler has already wired the `count` signal directly to the text node in the `<button>`, so only that tiny piece of the DOM updates.

KEY TAKEAWAY

The `<script>` block is not a render function. It is a one-time initialization script that sets up the local state, imports, and logic for a specific component instance.

Isn't this common sense?

The idea that a component's setup script should only run once may seem like obvious common sense, but in the modern framework landscape, it is not.

Because React components are functions that re-execute on every state change, developers often accidentally trigger infinite render loops. A developer might write a simple `console.log()` or a data fetch, only to open their DevTools and see their app printing 100s of errors in rapid repetition—looking like the app is on fire. In Svelte, because the `<script>` runs exactly once, these endless re-evaluation bugs are architecturally impossible.

INTERVIEW TIP

To impress the interviewer: Explicitly highlight the performance implications of this execution model. Because the `<script>` block only runs once, you can perform heavy initial calculations or object allocations inside it without fear of those operations re-running on every keystroke or interaction. This eliminates the need for **memoization**—a technique where developers are forced to manually cache expensive calculations so the framework remembers them instead of wasting CPU cycles re-doing the work on every render. While React developers rely heavily on complex hooks like `useMemo` or `useCallback` to patch this performance leak, Svelte's execution model makes it a non-issue.

Summary

The `<script>` tag encapsulates the JavaScript logic for a Svelte component. Because it runs precisely once during component creation, it provides a highly efficient environment for declaring state and functions without the runtime overhead of continuous re-evaluation during state changes.

	REACT COMPONENT (Runtime Render Function)	SVELTE <code><script></code> (Initialization Block)
Execution Frequency	Runs repeatedly on every state change	Runs exactly ONCE upon instantiation
Re-allocation Risk	High (functions/objects recreated every render)	Zero (declared once in memory)
Memoization Required?	Yes (heavy reliance on <code>useMemo</code> / <code>useCallback</code>)	No (inherently optimized by the COMPILER)

Lecture 10: TypeScript Integration

INTERVIEW QUESTION

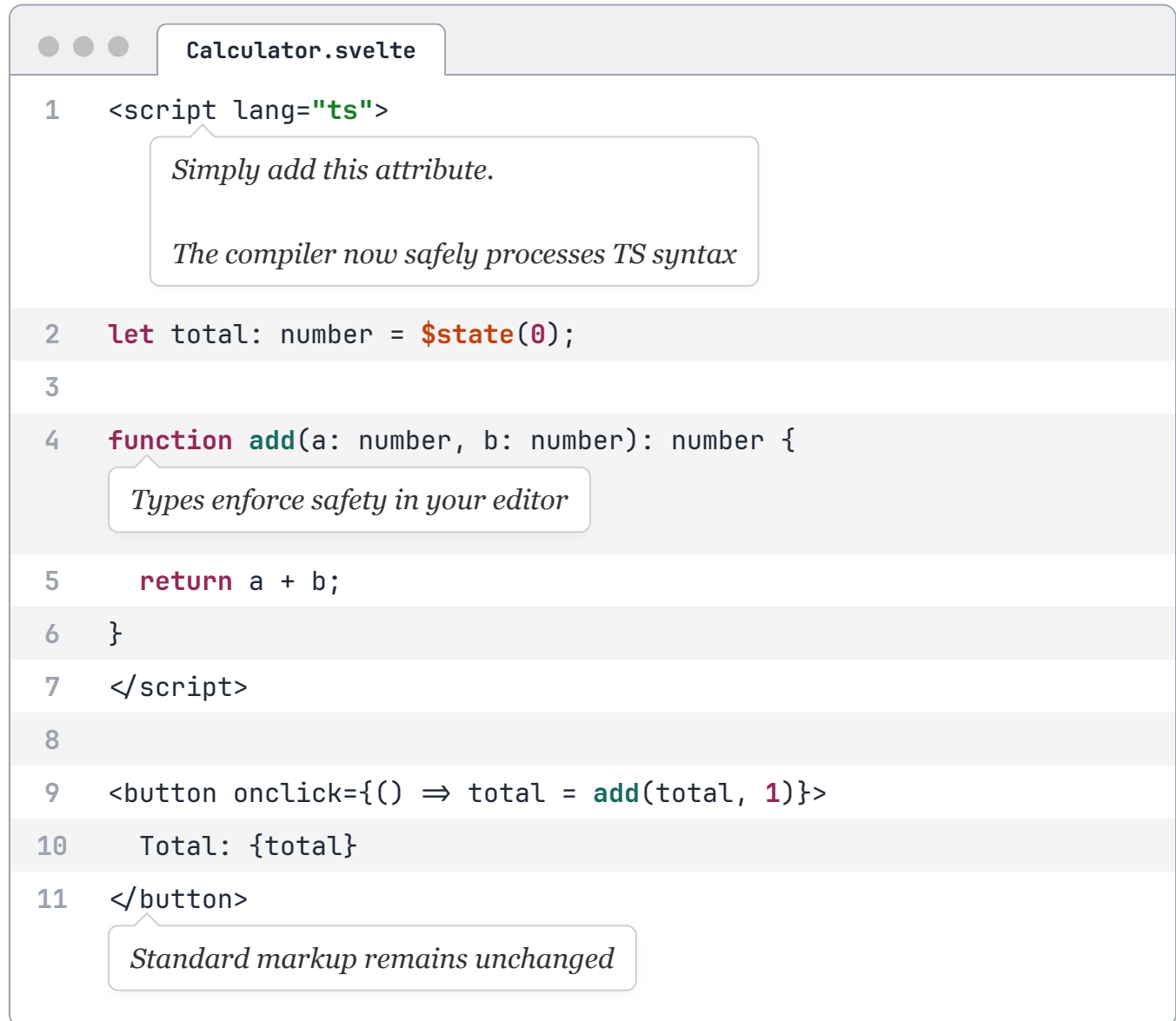
How do you enable and configure TypeScript natively inside a Svelte component?

TypeScript has become the industry standard for writing robust, maintainable web applications. Fortunately, modern Svelte projects (such as those generated by SvelteKit or Vite) come with native TypeScript support fully configured out of the box.

To enable TypeScript within a specific `.svelte` component, you do not need to install complex loaders or write custom Webpack configurations. You simply add a language attribute to your script tag.

The `lang="ts"` Attribute

By adding `lang="ts"` to the opening `<script>` tag, you instruct the Svelte compiler's preprocessor to expect TypeScript syntax instead of standard JavaScript.



The screenshot shows a code editor window titled "Calculator.svelte". The code is as follows:

```
1 <script lang="ts">
2   let total: number = $state(0);
3
4   function add(a: number, b: number): number {
5     return a + b;
6   }
7 </script>
8
9 <button onclick={() => total = add(total, 1)}>
10   Total: {total}
11 </button>
```

Annotations in the image:

- A callout box pointing to the `lang="ts"` attribute on line 1 says: "Simply add this attribute. The compiler now safely processes TS syntax".
- A callout box pointing to the type annotations on line 4 says: "Types enforce safety in your editor".
- A callout box pointing to the Svelte markup on line 11 says: "Standard markup remains unchanged".

When the Svelte compiler encounters a file with `lang="ts"`, a preprocessor intercepts the `<script>` block. Because browsers cannot execute TypeScript natively, the preprocessor strips away all the type annotations (`: number`, interfaces, generics) and converts the block back into plain, standard JavaScript. After the types are stripped, the standard Svelte compiler takes over to generate the reactive framework code.

KEY TAKEAWAY

Enabling TypeScript in Svelte is as simple as writing `<script lang="ts">` . The underlying preprocessor automatically strips the types during the build step, ensuring that only standard JavaScript ships to the browser.

INTERVIEW TIP

To impress the interviewer: It is vital to clarify what the compiler actually does. Many junior developers incorrectly assume that compiling a TypeScript Svelte component means the framework performs a massive, strict type-checking pass that slows down the build. In reality, Vite and Svelte simply *strip* the types (using fast tools like ESBuild). The actual deep type-checking is deferred to your IDE (like VS Code with the Svelte extension) or run as a separate continuous integration task via the `svelte-check` CLI tool.

Summary

TypeScript is enabled at the component level by adding the `lang="ts"` attribute to the script tag. This tells the build tools to process the block as TypeScript, allowing you to use static typing in your editor while the compiler safely strips the annotations away before generating the final browser-ready JavaScript bundle.

	STANDARD SCRIPT (<code><script></code>)	TYPESCRIPT SCRIPT (<code><script lang="ts"></code>)
Language Syntax	Vanilla JavaScript only	Full TypeScript support
Compiler Action	Directly compiles to reactive JS	STRIPS types, then compiles to reactive JS
Type Safety	None	Enforced by IDE and <code>svelte-check</code>

Lecture 11: The Default Export

INTERVIEW QUESTION

What is the default export of a compiled Svelte component file when imported into a JavaScript module?

When you write a `.svelte` file, you are writing a declarative mix of HTML, CSS, and JavaScript. The browser cannot natively understand this format. However, when the Svelte compiler processes your file, it transforms it into a highly optimized, standard JavaScript module.

The most important part of this transformation is what the compiler exposes to the rest of your application: **a default export**.

Svelte 5: The Setup Function

In Svelte 5, the default export of a compiled component is a plain JavaScript function (often referred to as a "setup" or "render" function).

When you import a component into another file, you are importing this compiled function. The function's job is to wire up the reactive signals (Runes), attach event listeners, and mount the specific DOM nodes required by that component.



The screenshot shows a code editor window titled "App.svelte". The code is as follows:

```
1 <script>
2 import ProfileCard from './ProfileCard.svelte';
3 </script>
4
5 <ProfileCard />
```

Annotations in the image:

- A callout box pointing to line 2: *Imports the compiled default function.*
- A callout box pointing to line 5: *You rarely invoke this function manually. Instead, Svelte's compiler handles the invocation when it sees the markup below.*
- A callout box pointing to line 5: *Svelte automatically invokes the imported function here*

KEY TAKEAWAY

A `.svelte` file is just the authoring format. The compiled output is a standard JavaScript module whose `default` export is a function that sets up the component's reactivity and DOM.

INTERVIEW TIP

To impress the interviewer: Highlight the architectural shift between Svelte 4 and Svelte 5. In Svelte 4, components compiled into JavaScript **Classes**, which had to be instantiated with `new Component(...)`. In Svelte 5, the compiler outputs plain **Functions**. This shift eliminates the memory overhead of class instantiation and aligns perfectly with Svelte 5's fine-grained, signal-based reactivity model, resulting in smaller bundles and faster execution.

Summary

The default export of a compiled Svelte 5 component is a standard JavaScript setup function. This function contains all the hardcoded DOM instructions and reactivity wiring necessary to bring your declarative `.svelte` markup to life in the browser.

	<code>.svelte</code> FILE (Authoring Format)	COMPILED JS MODULE (Browser Output)
Format	Declarative HTML/CSS/JS mix	Standard JavaScript Module
Default Export	(Not a valid JS module)	A Setup FUNCTION
Execution	Needs to be compiled	Invoked natively by the browser

Lecture 12: Programmatic Component Instantiation

INTERVIEW QUESTION

How do you instantiate a compiled Svelte component programmatically within vanilla JavaScript?

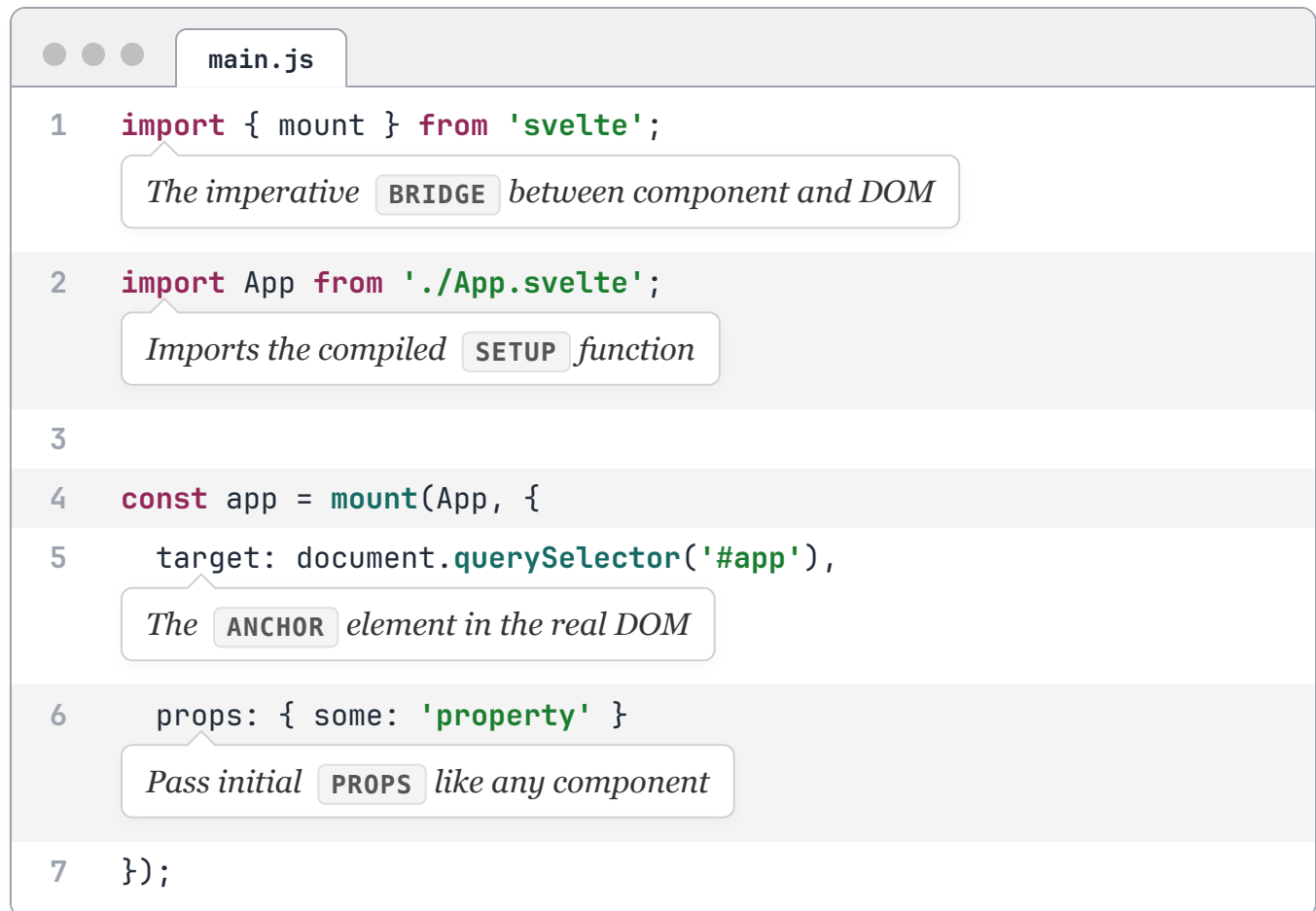
In Lecture 11 we established that the compiled output of a `.svelte` file is a standard JavaScript module whose default export is a plain function — the setup function that wires reactivity and builds the DOM. But a function sitting in a variable does not put pixels on a screen. You still need to answer the practical question: once I have imported this function into a vanilla JavaScript file, how do I actually make the component appear in the browser?

The gap between import and DOM

Imagine you are writing a plain `.js` file — not a `.svelte` component, not inside SvelteKit, not inside a Svelte markup template. You have imported a compiled component. The `import` gives you a function reference in memory. But there is no `<Component />` tag to write in a `.js` file, no markup compiler to rewrite your `import` into a DOM operation. You need to cross the gap from "I have a function" to "there is HTML on the page" by hand, using the imperative API Svelte provides.

The `mount` function

Svelte 5 ships a function called `mount` — the primary bridge between a compiled component function and the live DOM. You import `mount` from the `svelte` package, pass it the component function, and tell it which DOM element to fill.



`mount` does three things in one call. It instantiates the component — it runs the setup function you imported, which creates the reactive signals and builds the DOM subtree. It attaches that subtree into the live document at the element you passed as `target`. And it returns a component handle — an object you can use later for cleanup.

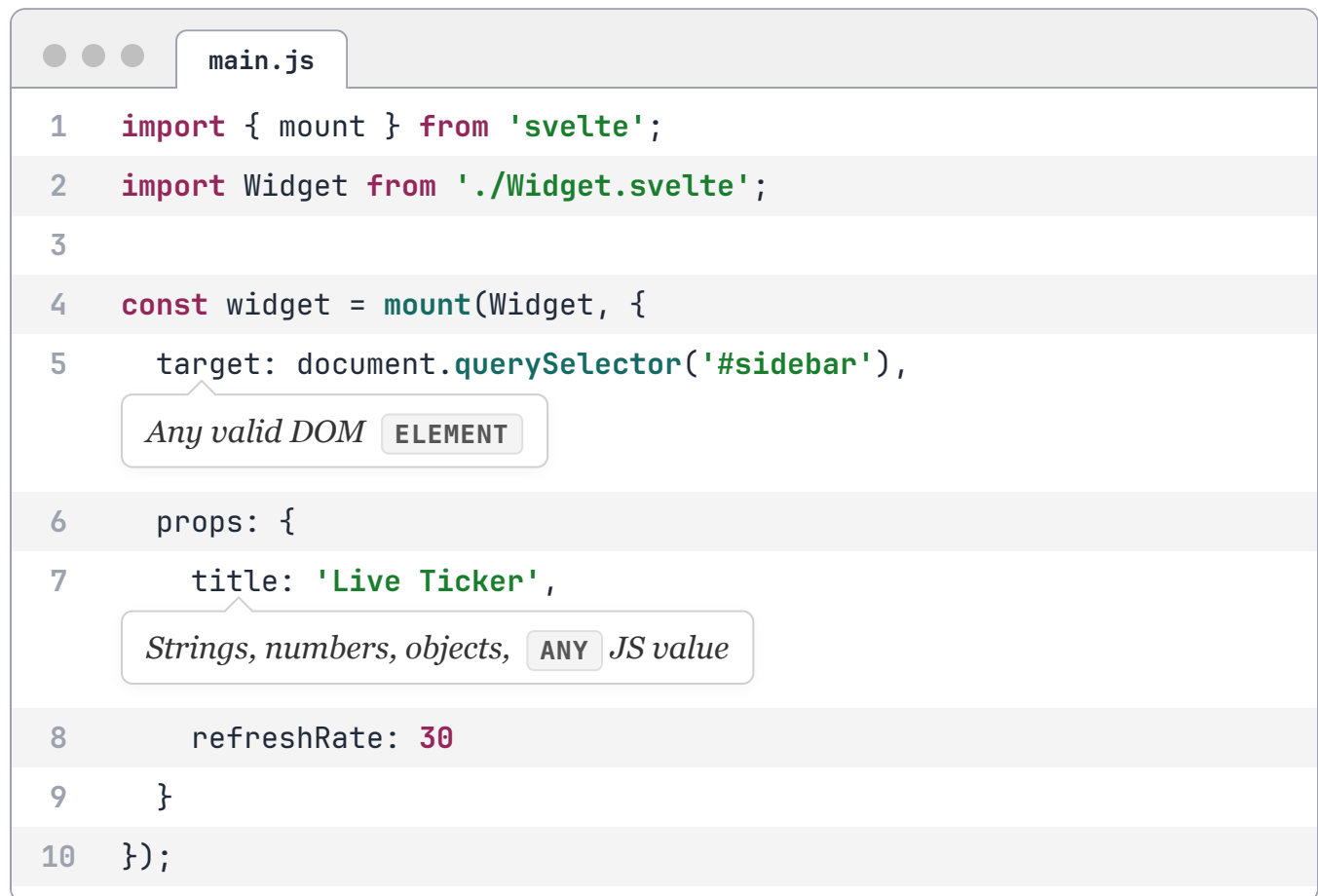
KEY TAKEAWAY

You do not call the compiled component function yourself. You hand it to `mount`, and `mount` calls it at the right moment, inside the right reactive context.

This is the essential inversion. The compiled function is not a "render this to a string" function you invoke and then inject. It is a setup function that expects to run inside Svelte's runtime tracking. Calling it directly, outside of `mount`, would create signals without an owning context — reactivity without a home. `mount` provides that home: it wraps the setup call so every `$state`, `$derived`, and `$effect` inside the component knows which component instance it belongs to.

The mount options object

The second argument to `mount` is an options object with two important properties. `target` is mandatory — it is the DOM element the component will be inserted into. The existing content of that element is replaced with the component's markup. `props` is optional — it is an object of initial values that become the component's props, exactly as if you had written `<App some="property" />` in Svelte markup.



```
1 import { mount } from 'svelte';
2 import Widget from './Widget.svelte';
3
4 const widget = mount(Widget, {
5   target: document.querySelector('#sidebar'),
6   props: {
7     title: 'Live Ticker',
8     refreshRate: 30
9   }
10 });
```

The screenshot shows a code editor window titled `main.js`. The code defines a `mount` function call. Two callout boxes provide additional context: one points to the `target` property value `document.querySelector('#sidebar')` with the text "Any valid DOM ELEMENT", and another points to the `title` property value `'Live Ticker'` with the text "Strings, numbers, objects, ANY JS value".

You can mount multiple components on the same page. Each call to `mount` creates an independent component instance with its own reactive scope, its own DOM subtree, and its own handle. A dashboard page might mount a chart component into one div, a news ticker into another, and a settings panel into a third — all from the same vanilla JavaScript file, with no enclosing Svelte root.

The component handle and `unmount`

The object returned by `mount` is not the component instance itself in the way Svelte 4 exposed it. It is an opaque handle — a token you keep so you can later tear the component down.

```
main.js
1  import { mount, unmount } from 'svelte';
2  import Modal from './Modal.svelte';
3
4  const modal = mount(Modal, {
5    target: document.querySelector('#overlay'),
6    props: { visible: true }
7  });
8
9
10 unmount(modal);
```

Later, when the user dismisses the modal

*Removes the component's **DOM** and tears down its reactivity*

`unmount` takes the handle and optionally an `outro` flag. When `outro` is true, any transition animations the component has declared will play before the DOM is removed, and `unmount` returns a Promise that resolves when those transitions finish.

```
main.js
1  await unmount(modal, { outro: true });
```

*Plays **TRANSITIONS** before removal, then resolves*

Without `outro`, the component is removed instantly — no animations, no promise, immediate DOM detachment.

INTERVIEW TIP

To impress the interviewer: Most candidates will describe `mount` as "the way to start a Svelte app." Show deeper understanding by explaining what `mount` returns and why it matters. The return value is not the component's internal state — it is a lightweight handle designed for one purpose: cleanup. Emphasise that every `mount` should be paired with an eventual `unmount`, especially in patterns like tooltip overlays or modal stacks where components are created and destroyed dynamically. Memory leaks in Svelte happen when you mount and forget to unmount.

Effects do not run during mount

One nuance that surprises developers coming from Svelte 4: when `mount` runs, effect callbacks — including `$effect` blocks and `onMount` — do not execute during the mount call itself. They are queued and flushed asynchronously, after the initial DOM is built. This is an intentional design choice that prevents cascading re-renders during setup.

If you need to force all pending effects to run immediately — common in testing, where you mount a component and then assert on the DOM — use `flushSync`.



```
1  import { mount, flushSync } from 'svelte';
2  import Counter from './Counter.svelte';
3
4  const counter = mount(Counter, {
5    target: document.querySelector('#test'),
6    props: { start: 0 }
7  });
8
9  flushSync();
```

*Forces pending **EFFECTS** to run immediately, for testing*

`flushSync` synchronously drains the effect queue. After it returns, every `$effect` block and every `onMount` callback in the mounted tree has run to completion. In production code you rarely need this — effects run soon enough for any real user interaction. In tests, it is essential.

The full imperative API surface

`mount` and `unmount` cover the client-side lifecycle. Svelte 5 provides two additional functions that complete the imperative API: `hydrate` for bringing server-rendered HTML to life, and `render` for producing HTML on the server in the first place.

`hydrate` is the SSR-aware sibling of `mount`. It takes the same arguments — a component, a target, and optional props — but instead of building fresh DOM from scratch, it scans the existing HTML inside the target, reuses every DOM node it finds, and wires reactivity onto the already-built structure.



```
main.js
1  import { hydrate } from 'svelte';
2  import App from './App.svelte';
3
4  const app = hydrate(App, {
5    target: document.querySelector('#app'),
6    props: { some: 'property' }
7  });
```

Reuses server-rendered HTML already in the page

`render` runs on the server, not the browser. It takes a component and props and returns an object with `body` and `head` properties — strings of HTML you can insert into a server response.

● ● ● server.js

```
1  import { render } from 'svelte/server';
   Imported from 'svelte/server', not 'svelte'
2  import App from './App.svelte';
3
4  const result = render(App, {
5    props: { some: 'property' }
6  });
7  result.body;
   HTML for inside the <body> tag
8  result.head;
   HTML for inside the <head> tag (meta, links, title)
```

KEY TAKEAWAY

`mount` builds fresh DOM and attaches it. `hydrate` reuses existing DOM and wires reactivity onto it. `render` never touches a DOM — it produces strings.

The Svelte 4 contrast: from Classes to functions

In Svelte 4, the imperative API looked fundamentally different. The compiled component was a JavaScript class, and you instantiated it with the `new` keyword.

```
1  import App from './App.svelte';
2
3  const app = new App({
4    target: document.querySelector('#app'),
5    props: { name: 'world' }
6  });
7
8  app.$destroy();
```

Svelte 4 class-based **INSTANTIATION**

Svelte 4 **CLEANUP** *method*

The shift from classes to functions in Svelte 5 is not cosmetic. Classes carry prototype chains, constructor overhead, and instance-property dictionaries — all of which add weight to every component instance. Functions have none of that. The `mount(App, ...)` pattern is lighter, treeshakable, and aligns with the broader JavaScript ecosystem's movement toward functions over classes.

The cleanup story also changed. In Svelte 4, every component instance had a `.$destroy()` method. In Svelte 5, the handle returned by `mount` has no methods — you pass it to the standalone `unmount()` function. This separation makes the handle a pure data token, not an object with behaviour, which is easier to serialise, pass across boundaries, and garbage-collect when no longer referenced.

Summary

To instantiate a compiled Svelte component from vanilla JavaScript, import `mount` from `svelte` and pass the component function to it alongside a `target` DOM element. `mount` returns an opaque handle you can later pass to `unmount` for cleanup. Effects do not run during `mount` — use `flushSync` if you need them immediately. For server-rendered pages, use `hydrate` instead of `mount` to reuse existing HTML. For server-side rendering, use `render` from `svelte/server` to produce HTML strings. The imperative API in Svelte 5 replaces the class-based `new Component()` pattern from Svelte 4 with a lighter, functional design.

	SVELTE 4 (Legacy)	SVELTE 5
Import	<pre>import App from './App.svelte'</pre>	<pre>import App from './App.svelte'</pre>
Instantiate	<pre>new App({ target, props })</pre>	<pre>mount(App, { target, props })</pre>
Cleanup	<pre>app.\$destroy()</pre>	<pre>unmount(app)</pre>

Server-side	(Compiler option)	<code>render(App, { props })</code> from <code>svelte/server</code>
SSR Hydration	<code>new App({ hydrate: true })</code>	<code>hydrate(App, { target, props })</code>
Underlying type	JavaScript Class	Plain JavaScript Function

Lecture 49: Syncing Local Variables with Incoming Props

INTERVIEW QUESTION

How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

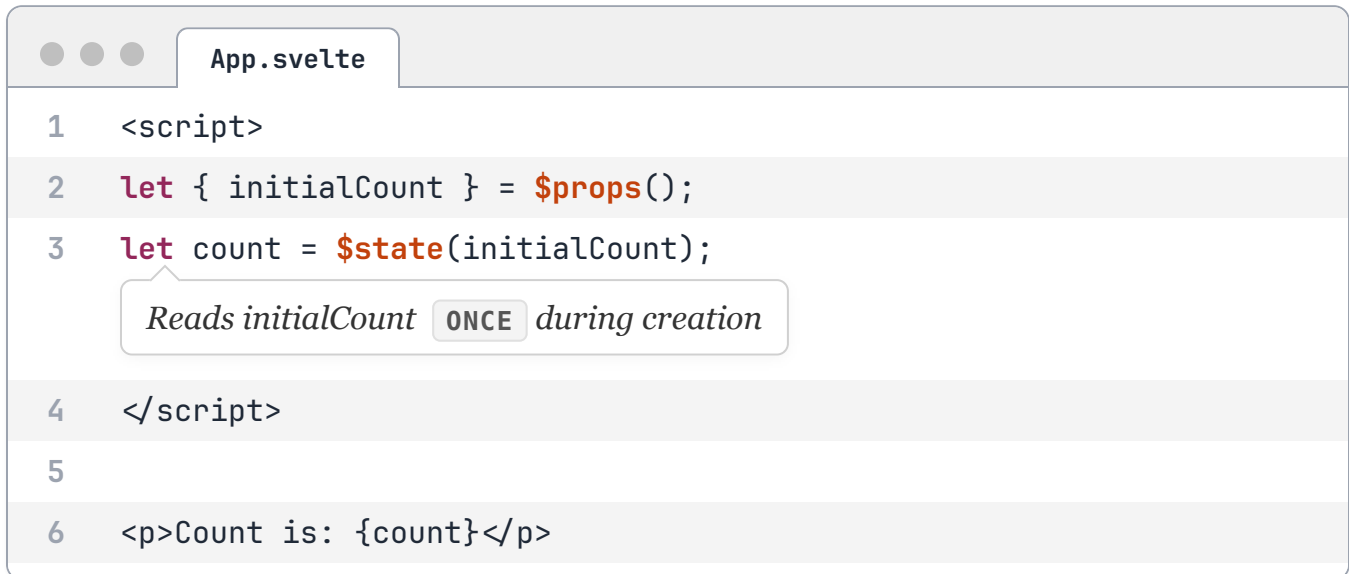
A common source of confusion when moving from legacy Svelte to Svelte 5 runes is how to handle a local variable that needs to initialize from an incoming prop, but also needs to stay in sync when that prop changes over time.

If you pass a prop down to a child component and assign it to a `$state` rune — the rune used to declare reactive state — it will capture the *initial* value of that prop perfectly. However, if the parent later updates the prop, the child's `$state` variable will stubbornly hold onto the original value. The connection is severed the moment the child component is created.

To solve this, we rely on the `$derived` rune — a reactive declaration that automatically recalculates its value whenever its dependencies change.

The problem with \$state

When a component is instantiated, code inside the `<script>` block runs exactly once.



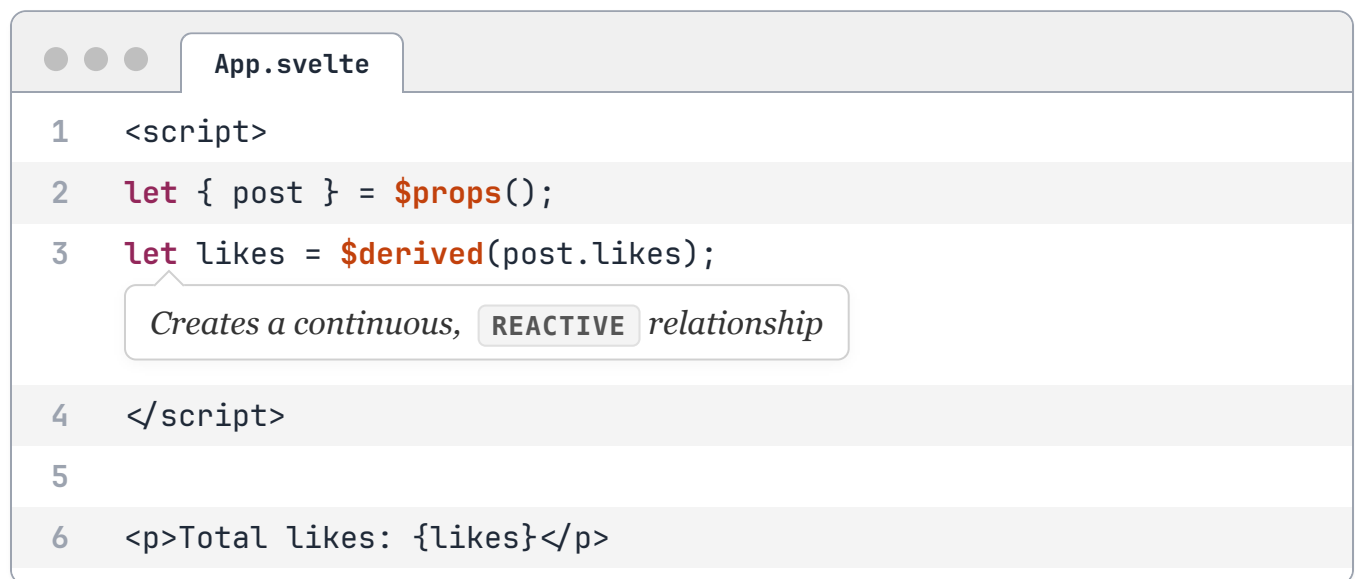
```
1  <script>
2  let { initialCount } = $props();
3  let count = $state(initialCount);
4  </script>
5
6  <p>Count is: {count}</p>
```

Reads initialCount ONCE during creation

In the example above, `count` is a completely independent variable. It took `initialCount` as its starting value, but it has no ongoing relationship with it. If the parent component changes `initialCount`, the child component will ignore the change, and `count` will not update.

The solution: `$derived`

If you want a variable to dynamically sync with a prop, you declare it as derived state. Anything read synchronously inside a `$derived` expression is considered a **dependency** — a reactive value that Svelte watches for changes. When the dependency changes, the derived state is marked as **dirty** (meaning it's stale and needs recalculation) and will be updated automatically.



```
1 <script>
2 let { post } = $props();
3 let likes = $derived(post.likes);
4 </script>
5
6 <p>Total likes: {likes}</p>
```

*Creates a continuous, **REACTIVE** relationship*

Because `$derived` expressions are re-evaluated when their inputs change, `likes` will always stay in sync with `post.likes`. Whenever the parent updates the `post` prop, `likes` recalculates.

Overriding derived values for Optimistic UI

Historically, derived values were strictly read-only. You could not assign a new value to `likes = 5`. But starting in Svelte 5.25, you can temporarily override the value of a `$derived` variable by simply reassigning it (as long as you didn't declare it with `const`).

This is specifically designed for **optimistic UI** — a pattern where you update the user interface immediately before the server confirms the change, making the app feel lightning fast.

Here is how it works in practice:

● ● ● App.svelte

```
1 <script>
2 let { post, like } = $props();
3 let likes = $derived(post.likes);
4
5 async function onclick() {
6   likes += 1;
7
8   try {
9     await like();
10
11    } catch {
12      likes -= 1;
13    }
14  }
15
16 </script>
17
18 <button {onclick}>❤️ {likes}</button>
```

Derived from the **SOURCE** of truth (the prop)

1. optimistic override: increment **LOCALLY** immediately

2. tell the server to **SAVE** the change, re-deriving likes

3. rollback: if network request failed, **REVERT** the override

When you write `likes += 1`, you are temporarily overriding the derived value. When the parent eventually passes down a new `post` object with updated likes, the `$derived` rune recalculates, and the override is washed away by the new source of truth.

Summary

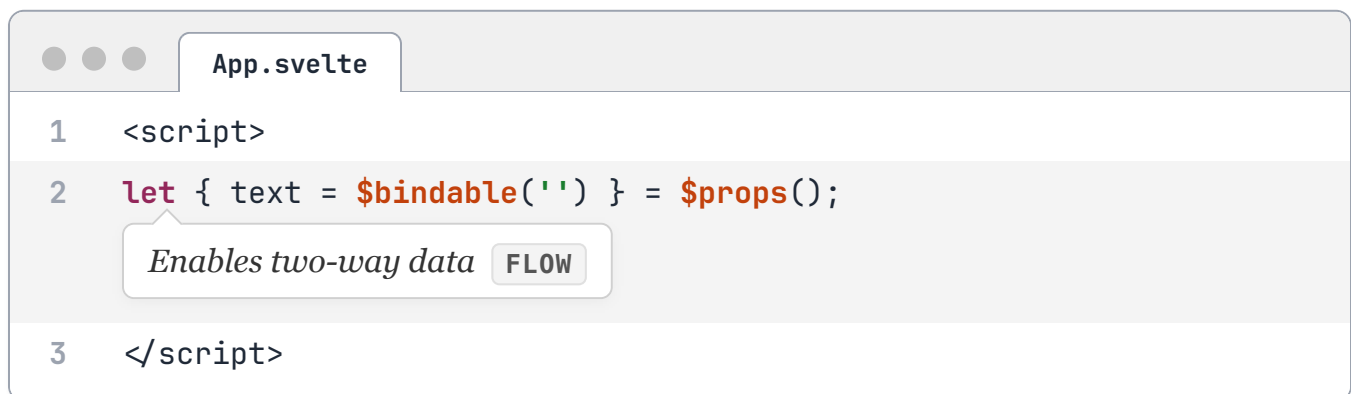
If you need a local variable that dynamically syncs with an incoming prop, do not use `$state`. Use `$derived`. And if you need to let the user temporarily update that value before the parent data catches up, you can safely reassign the derived variable.

Beyond the basics

Frequency Rating System ★★★ = Very common
gotcha / pattern ★★☆☆ = Common gotcha / pattern
★☆☆ = Niche / Advanced pattern

★★★ Two-way prop sync uses `$bindable`

The `$derived` pattern above is one-way (parent → child). When the child must also push changes back up to the parent — a text input bound to a parent-owned string, a checkbox toggling parent state — declare the prop with `$bindable()` and use `bind:text` in the parent. The Svelte docs are explicit: "don't mutate props. Either use callback props to communicate changes, or — if parent and child should share the same object — use the `$bindable` rune."



```
1 <script>
2 let { text = $bindable('') } = $props();
3 </script>
```

Enables two-way data FLOW

★★☆ Multi-statement derived values use `$derived.by`

The `$derived(expr)` form takes a single expression. If your transform needs a loop, an intermediate `let`, or a `try/catch`, switch to `$derived.by()`. Under the hood `$derived(expr)` is exactly equivalent to `$derived.by(() => expr)`; `.by` exists only for multi-statement bodies.

● ● ● App.svelte

```
1 <script>
2   let formatted = $derived.by(() => {
3     let str = text.trim();
4     return str.toUpperCase();
5   });
6 </script>
```

Allows complex LOGIC inside the derivation

☆☆☆ Editable-and-syncing local drafts use `$effect`

When the child must own its own edits (a form draft the user is typing into) but also needs to re-sync when the parent hands down a new prop, the pattern is `$state(prop.value)` combined with `$effect()`. Use this sparingly —

`$effect`-driven sync can create loops if you write back to the same signal you read (see Lecture 21). Default to `$derived` first; reach for this only when editability and continuous sync are both required.

● ● ● App.svelte

```
1 <script>
2 let { user } = $props();
3 let draft = $state(user.name);
4
5 $effect(() => {
6   draft = user.name;
7 });
8 </script>
```

Carefully syncs **UPSTREAM** changes into local draft

★★★ Destructuring a prop is a one-shot snapshot

`let { title } = $props()` reads `title` once, at component setup. Later changes to the parent's prop do not flow into the destructured `title` variable. This is the same JavaScript-level gotcha as destructuring a `$state` proxy (see Lecture 50). If you need ongoing reactivity, read through the props object: `{title}` in the template, or `let p = $props()` and then `p.title`.



```
1 <script>
2 let { title } = $props();
3 </script>
```

*Severs the **REACTIVE** connection to the prop*

★★☆ Derived expressions must be side-effect-free

The `$derived` body may not mutate state. Svelte disallows this because derived values must be pure functions of their inputs — side effects belong in `$effect`, not `$derived`. If your "sync" needs to write to another signal as part of its work, you are writing an effect, not a derived.

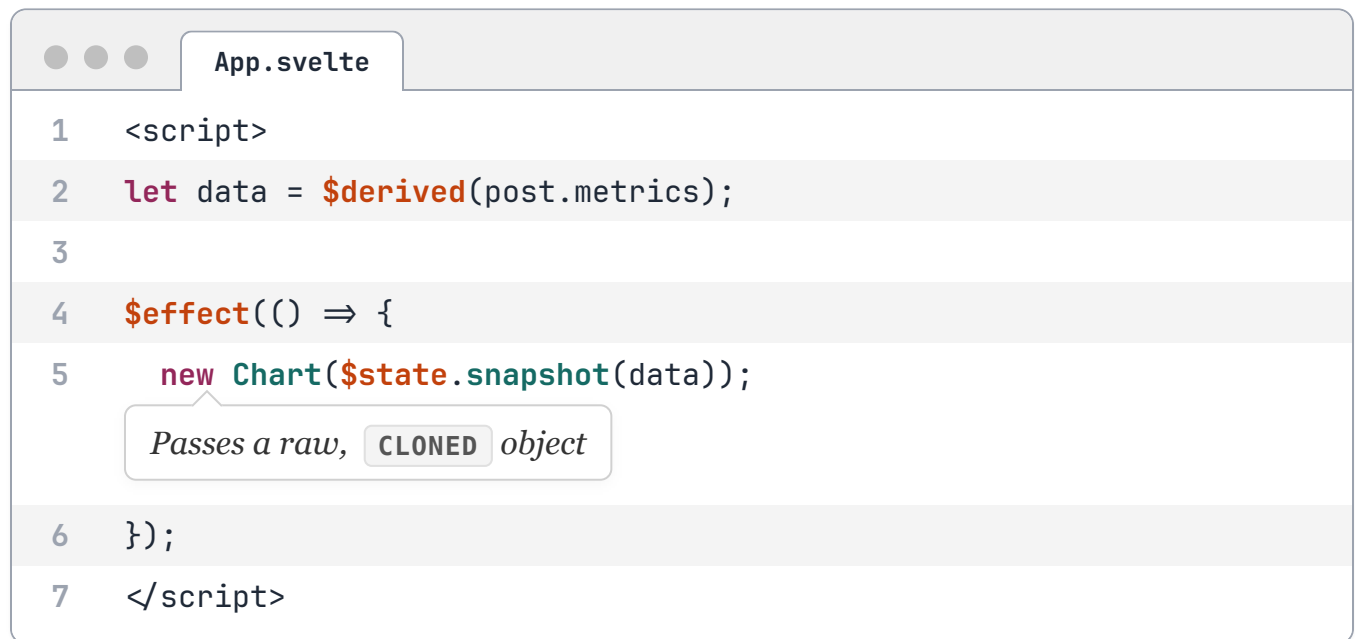
● ● ● App.svelte

```
1 <script>
2 let count = $state(0);
3 let doubled = $derived.by(() => {
4   count += 1;
5   return count * 2;
6 });
7 </script>
```

Mutating state here is a **COMPILE ERROR**

☆☆☆ Passing proxied data to external libraries

A `$derived` whose source is a `$state` proxy returns a proxied value. Some libraries (structured clone, certain chart libraries) trip over Proxy objects. Wrap the handoff with `$state.snapshot(derivedValue)` to pass a static deep clone instead.



```
1 <script>
2 let data = $derived(post.metrics);
3
4 $effect(() => {
5   new Chart($state.snapshot(data));
6 });
7 </script>
```

*Passes a raw, **CLONED** object*

★★★ Don't conflate the three syncing runes

Picking the wrong one is the most common source of "why isn't my UI updating" bugs in Svelte 5 components that receive props.

● ● ● App.svelte

```
1 <script>
2 let { data } = $props();
3 let a = $state(data);
4 let b = $derived(data);
5 let { c = $bindable() } = $props();
6 </script>
```

Initializes once, then goes **DEAF**

Listens forever, but is **READ-ONLY**

Listens forever AND is **WRITABLE**

Lecture 50: Array Updates in \$state vs Legacy Re-assignment

One of the most common stumbling blocks for developers learning Svelte 4 was updating an array. If you had a list of items and you called `items.push(newItem)`, the array would update in memory, but the user interface would completely ignore the change. The new item simply would not appear on screen. This failure occurred because legacy Svelte's reactivity system was fundamentally tied to the **assignment operator** — the literal `=` sign in your code. If the compiler did not see an `=` sign targeting the `items` variable, it assumed the variable had not changed, and therefore no UI update was scheduled. To fix this, developers were forced to write repetitive, non-intuitive boilerplate like `items = [...items, newItem]` or `items = items` simply to force the compiler to notice the mutation.

Svelte 5 eliminates this entire class of bugs by abandoning assignment-based tracking in favor of the `$state` rune. When you pass an array into `$state()`, Svelte does not just hold onto the array; it transforms it into a deeply reactive **state proxy** — a native JavaScript object wrapper that intercepts every time you read or write a property underneath it. Because the proxy intercepts operations at runtime rather than relying on compile-time source code analysis, Svelte 5 inherently understands when you call mutating array methods like `.push()`, `.pop()`, or `.splice()`. When the proxy detects that the array has been mutated through one of these methods, it automatically alerts the reactivity system to surgically update the specific parts of the UI that depend on that array.

● ● ● App.svelte

```
1 <script>
2 let items = $state([1, 2]);
3
4 function addItem() {
5   items.push(3);
6 }
7 </script>
```

Intercept array method mutations **AUTOMATICALLY** at runtime

This shift from compile-time assignment tracking to runtime proxies drastically simplifies how you work with complex data structures. In Svelte 5, you write standard, idiomatic JavaScript. To add an item to an array, you just call `items.push(newItem)`. To update a specific item deep inside an array of objects, you simply access it by index and reassign its property, such as `todos[0].done = true`. Because `$state` creates deep proxies by default, every nested object and array inside your root state is also recursively converted into a proxy, ensuring that even deeply nested mutations are caught and tracked automatically without requiring full object replacement.

● ● ● App.svelte

```
1 <script>
2 let todos = $state([ { text: 'Buy milk', done: false } ]);
3
4 function completeTask() {
5   todos[0].done = true;
6 }
7 </script>
```

Deeply nested mutations are **CAUGHT** without replacement

However, the proxy-based system does introduce a specific constraint regarding destructuring. If you destructure a primitive value out of a state proxy, such as `let { done } = todos[0]`, you are pulling out the raw boolean value at that exact moment in time, completely severing its connection to the reactive proxy. If you later execute `todos[0].done = true`, your local `done` variable will not magically update, because standard JavaScript passes primitive values by value, not by reference. Therefore, when working with `$state` arrays, you must avoid destructuring primitive properties if you need them to remain reactive over time; instead, reference the proxy properties directly in your templates and logic.

App.svelte

```
1 <script>
2 let todos = $state([{ text: 'Buy milk', done: false }]);
3
4 function toggle() {
5   let { done } = todos[0];
6     done = true;
7 }
8 </script>
```

Pulls raw primitive, severing CONNECTION to proxy

Original proxy is NOT UPDATED